

# The Machine Learning Development Process

*Error Analysis, Adding Data, Transfer Learning, Deployment, Fairness, and Precision–Recall*

Rokan Uddin Faruqui
Professor, Department of CSE

University of Chittagong
rufaruqui@cu.ac.bd

Week 6 · Part 2

# From diagnosis to a development process

**Part 1 gave us**

- Train / cross-validation / test: fit, choose, report.
- Bias gap and variance gap, measured against a baseline.
- Learning curves: does more data help?

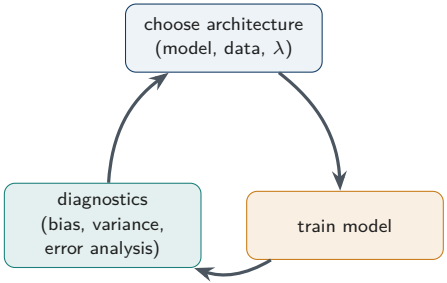
Bias and variance say *how much* a model is wrong. They do not say *which* examples it gets wrong, where more data should come from, or whether the model should be deployed at all. Today is about the rest of the process.

**Today**

1. The iterative loop, and error analysis
2. Adding data: targeted collection, augmentation, synthesis
3. Transfer learning
4. The full cycle of a project; fairness, bias and ethics
5. Skewed classes: precision, recall and $F_1$

# The Iterative Loop and Error Analysis

# The iterative loop of ML development




**Example: a spam classifier**

Features: $x_j = 1$ if word $j$ of a 10 000-word list appears in the email. Ideas for improving it:

- collect more data ("honeypot" accounts that attract spam);
- features from the email's routing headers;
- features from the body: punctuation, "w4tches" for "watches";
- algorithms to detect deliberate misspellings.

**Key idea**

Every turn of the loop costs time. Good diagnostics make each turn count.

# Error analysis: look at the mistakes

## Illustrative: 500 of 5000 validation emails misclassified

Pick 100 of the 500 at random and tag each by hand:

| category | count |
|---|---|
| pharmaceutical spam | 21 |
| phishing (stealing passwords) | 18 |
| unusual email routing | 7 |
| spam inside an embedded image | 5 |
| deliberate misspellings | 3 |

Categories may overlap: one email can be both pharmaceutical and misspelt.

## What the table decides

A perfect misspelling detector would fix at most 3 in 100 mistakes — about 15 of the 500. Pharmaceutical spam and phishing together account for 39: that is where effort should go (more pharmaceutical examples; features for URLs in phishing emails).

Why 100? Enough to see the large categories clearly; few enough to read in an hour or two.

## Limits

Easy for tasks people do well (reading email, recognising objects); much harder when people cannot judge either, such as predicting which advert a user will click.

# Adding Data

# More data — of the right kind

## Targeted collection

"More of everything" is slow and expensive. If error analysis says pharmaceutical spam is the problem, collect *pharmaceutical spam*: for instance, search the unlabelled mail you already have for it and label only that.

## Two views of a project

**Model-centric**: hold the data fixed, improve the algorithm.

**Data-centric**: hold a good algorithm fixed, improve the data. With strong off-the-shelf networks, the second is often the faster route to better performance.

## Three ways to add data

1. **Collect** more real examples, targeted by error analysis.
2. **Augment**: create new examples by modifying existing ones, keeping the label.
3. **Synthesise**: create new examples from scratch.

## Key idea

All three serve a model with *high variance*. Part 1's learning curve tells you whether that is the model you have.

# Data augmentation

One labelled “A” becomes many






random pixel noise: usually useless

## Other domains

**Speech**: add background noise — a crowd, a car, a bad phone line — to a clean recording. **Text**: replace words with synonyms. Each modified copy keeps the original label.

## The rule

A distortion helps only if it represents variation you expect in the *test* set. Rotated letters and noisy calls occur in practice; per-pixel random noise does not, and adds little.

## Do not change the label

Rotate a “6” by 180° and it is a “9”. Augmentations must preserve what the label means.

# Data synthesis

## Photo OCR

Reading text in photographs of shop signs and number plates. Real labelled crops are scarce. Synthesise them: render characters in many fonts, sizes, colours and contrasts, on varied backgrounds, and crop.



synthetic training images of the letter “A”

## When synthesis works

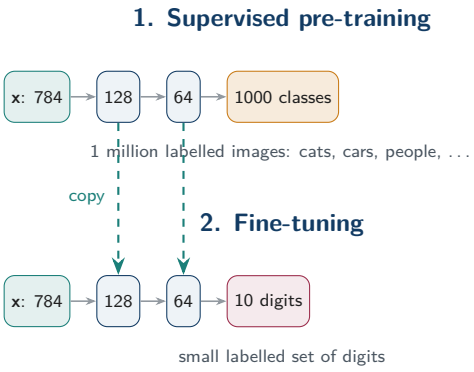
When you can write a program whose output looks like real inputs, with the label known by construction: rendered text, simulated sensor readings, game-engine images for self-driving.

## The gap between synthetic and real

A model can learn the renderer’s quirks instead of the task. Always validate on *real* data, and check with error analysis that the synthetic data fixes the categories it was made for.

# Transfer Learning

# Using data from a different task




## Two options for step 2

**Option 1**: train only the new output layer; keep the copied layers fixed. Best when the new data set is very small.

**Option 2**: train every layer, starting from the copied values. Better when you have more data.

## Parameters

Copied: $100\,480 + 8\,256 = 108\,736$. New output layer: $64 \cdot 10 + 10 = 650$ (it replaces 65 000). Option 1 trains 650 of 109 386.

### Generic early features

Week 4: the early layers of an image network detect edges, then corners and curves, then parts. These are useful for almost *any* image task, so layers learned on a million photographs give a head start on a few hundred X-rays or digits.

### Same type of input

Pre-train on images for an image task, on audio for a speech task, on text for a language task. A network pre-trained on photographs offers nothing to a spreadsheet of loan applications.

### In practice

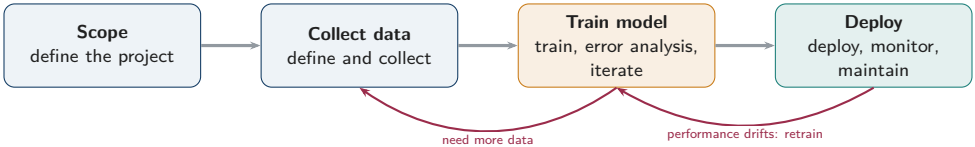
1. Download a network pre-trained on a large data set with the same input type — often published by researchers or companies.
2. Replace the output layer to suit your classes.
3. Fine-tune on your own data (option 1 or 2).

### The same idea, at scale

Large language models such as GPT and BERT are pre-trained on vast amounts of text, then fine-tuned for a specific task — including tasks in Bangla.

# The Full Cycle, and Fairness

# The full cycle of a machine learning project




## Deployment

The model runs on an **inference server**. A mobile app sends an input — a voice query, a transaction — through an API call and receives a prediction. Engineering concerns then dominate: reliable prediction at scale, logging inputs and outputs, monitoring, and updating the model.

## MLOps

Machine-learning operations: the practice of building, deploying and maintaining ML systems. Logged data from deployment is often the best new training data — and the first sign that the world has changed (**data drift**).

## When systems have done harm

- A hiring tool that learned to rank women's applications lower, from past hiring decisions.
- Face recognition that is markedly less accurate for darker-skinned faces.
- Loan approval that disadvantages groups under-represented in the training data.

## Adverse uses

Deepfakes made without consent; recommender systems that spread toxic content because it is engaging; fraud and fake reviews generated at scale.

## Bias in, bias out

"Bias" here is not the bias of Part 1. A model trained on historical decisions learns the patterns in those decisions — including unfair ones — and applies them at scale.

## Practical guidelines

1. A diverse team brainstorms what could go wrong, especially for vulnerable groups.
2. Search the literature for standards in your field.
3. Audit for harm *before* deployment, e.g. accuracy per group.
4. Plan mitigation (e.g. roll back) and keep monitoring.

# Skewed Classes: Precision and Recall

# When accuracy misleads

## Screening for a rare disease

1000 patients in the validation set, 25 of them ill ($y = 1$).

- A trained classifier: accuracy 98.5%.
- `print("y = 0")` for everyone: accuracy 97.5%.

The program that never finds a single patient is nearly as "accurate".

## Confusion matrix of the classifier

| | actual 1 | actual 0 |
|---|---|---|
| predicted 1 | TP = 15 | FP = 5 |
| predicted 0 | FN = 10 | TN = 970 |

## Precision and recall

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{15}{20} = 0.75$$

of the patients we flagged, the fraction actually ill.

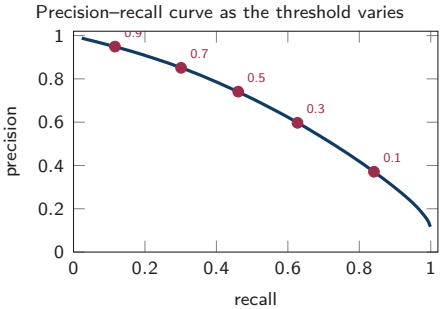
$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{15}{25} = 0.60$$

of the ill patients, the fraction we found.

## Key idea

"Always $y = 0$" has recall 0 and an undefined precision (0/0): the two numbers expose it immediately. Convention: $y = 1$ is the rare class.

# Trading precision for recall with the threshold




Illustrative: 10% positives, scores $\mathcal{N}(2,1)$ vs. $\mathcal{N}(0,1)$; labels are thresholds on $P(y{=}1 \mid \mathbf{x})$.

**Predict $\hat{y} = 1$ if $f(\mathbf{x}) \geq$ threshold**

| threshold | precision | recall | $F_1$ |
|---|---|---|---|
| 0.1 | 0.371 | 0.841 | 0.515 |
| 0.3 | 0.597 | 0.627 | **0.612** |
| 0.5 | 0.741 | 0.461 | 0.568 |
| 0.7 | 0.851 | 0.301 | 0.444 |
| 0.9 | 0.949 | 0.116 | 0.206 |

**Raise** it: predict 1 only when confident — higher precision, lower recall (before an invasive treatment). **Lower** it: miss fewer cases — higher recall, lower precision (when a miss is dangerous and a follow-up test is cheap).

**A choice, not a fit**

Set the threshold from the costs of the two kinds of error.

# Combining precision and recall: the $F_1$ score

**Three algorithms — which is best?**

| | $P$ | $R$ | average | $F_1$ |
|---|---|---|---|---|
| algorithm 1 | 0.5 | 0.4 | 0.45 | **0.444** |
| algorithm 2 | 0.7 | 0.1 | 0.40 | 0.175 |
| algorithm 3 | 0.02 | 1.0 | **0.51** | 0.0392 |

Algorithm 3 predicts $y = 1$ for almost everyone: perfect recall, useless precision. The plain average ranks it first.

**$F_1$ score**

$$F_1 = \frac{1}{\frac{1}{2}\left(\frac{1}{P} + \frac{1}{R}\right)} = \frac{2PR}{P + R}$$

The **harmonic mean** of $P$ and $R$. It is dominated by the smaller of the two, so a classifier must do well on both to score well.

**Check:** algorithm 1, $2(0.5)(0.4)/0.9 = 0.444$; the screening classifier, $2(0.75)(0.6)/1.35 = 0.667$.

In the threshold table, $F_1$ peaks at about 0.31 ($F_1 = 0.612$) — not at the default 0.5.

# Wrap-up and Assessment

# Summary

## What to remember

- Development is a loop: choose, train, diagnose. Diagnostics decide the next turn.
- Error analysis: tag a sample of the mistakes by hand; fix the largest categories first.
- Add data where error analysis points; augment with realistic distortions; synthesise when you can render the input.
- Transfer learning: pre-train on a large related task, replace the output layer, fine-tune.
- Deploy, monitor and retrain; audit for harm before and after.
- Skewed classes: use precision, recall and $F_1$, not accuracy.

## Formulas

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

$$F_1 = \frac{2PR}{P + R}$$

## Next

Module 3, Week 7: unsupervised learning — clustering with $k$-means, and anomaly detection.

## Descriptive questions

1. Describe the iterative loop of ML development and explain the role of diagnostics in it. **[4]**
2. Explain how to carry out an error analysis. Using the spam table, explain why a misspelling detector is a poor use of effort. **[6]**
3. Distinguish collecting, augmenting and synthesising data, with one example of each. When will none of them help? **[6]**
4. State the rule for choosing data augmentations, and give one augmentation that would harm a digit classifier. **[5]**
5. Explain transfer learning, its two fine-tuning options, and when to use each. Why must the pre-training input be of the same type? **[6]**
6. Describe the four stages of the full cycle of an ML project, and two reasons a deployed model may need retraining. **[5]**

## Descriptive questions (continued)

7. Give two examples of ML systems that caused harm, and four guidelines for reducing the risk. **[5]**
8. Explain why accuracy is misleading for skewed classes, using a classifier that always predicts $y = 0$. **[4]**
9. A fraud model on 10 000 transactions gives TP $= 40$, FP $= 60$, FN $= 20$. Compute precision, recall, $F_1$ and accuracy. **[6]**
10. Explain how raising the threshold of a logistic classifier changes precision and recall, and sketch the precision–recall curve. **[6]**
11. Why is $F_1$ preferred to the arithmetic mean of $P$ and $R$? Compute both for $P = 0.9$, $R = 0.2$. **[5]**
12. For transfer learning from a $784 \to 128 \to 64 \to 1000$ network to a 5-class task, how many parameters does option 1 train? **[4]**

## Design questions

1. **Design an error analysis.** A Bangla handwritten-address reader for the postal service misreads 8% of envelopes. Design the error analysis: sample size, categories you expect, and how each finding would change what you do next. **[12]**
2. **Design an augmentation pipeline.** For a rice-leaf disease classifier trained on photos from a few fields, design augmentations that reflect the conditions farmers' phones will meet. Name one tempting augmentation you would reject, and why. **[10]**
3. **Design a threshold.** A dengue screening model flags patients for a blood test costing Tk 500; a missed case costs about Tk 50 000 in later treatment. Design how to choose the threshold from validation data, and state what you report to the hospital. **[12]**
4. **Design a fairness audit.** A bank will use a model to pre-screen micro-loan applications from rural and urban applicants. Design the audit you would run before deployment, the metric per group, and the condition that would stop the launch. **[10]**

# Reading

**Primary text**

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§4.4.2** — confusion matrices, changing the threshold, and the ROC curve; **§10.3.4–10.3.5** — data augmentation and using pre-trained networks.

**Supplementary**

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§1.5** — decision theory: unequal error costs and the decision threshold; **§5.5.3** — invariances learned from transformed training data.

Lab (CSE 816, Week 6): model evaluation and selection; diagnosing bias and variance.